

Practical No. 10

ON

SSCA4030 - Image Processing

TITLE: Apply the Hough Transform for Line/Circle Detection

BACHLEOR OF SCIENCE IN INFORMATION TECHNOLOGY
(HONOURS.)

SUBMITTED BY:

Name: Shah Rishabh Alpeshbhai

Enrolment: 23SS02IT181

Class: BSCIT-H – 7A - Batch 2023-27

The **Hough Transform** is a feature-extraction technique used to detect geometric shapes in an image.

It is commonly used for:

- Straight-line detection
- Circle detection
- Ellipse detection
- Other parametric shapes

The basic image-processing workflow is:

Input Image → Grayscale → Noise Reduction → Edge Detection → Hough Transform → Detected Shapes

Hough Transform for Line Detection

A straight line can be represented using:

$$y = mx + c$$

However, this equation has a problem with vertical lines because their slope approaches infinity.

Therefore, the Hough Transform commonly uses the polar representation:

$$\rho = x \cos \theta + y \sin \theta$$

Where:

- **ρ (rho)** = perpendicular distance from the origin to the line
- **θ (theta)** = angle of the perpendicular from the origin
- **x, y** = coordinates of an edge pixel

Each edge pixel votes for possible (ρ, θ) combinations. A large number of votes at a particular combination indicates the presence of a line.

Hough Transform for Circle Detection

A circle is represented as:

$$(x - a)^2 + (y - b)^2 = r^2$$

Where:

- **a** = x-coordinate of the center
- **b** = y-coordinate of the center
- **r** = radius

The Hough Circle Transform searches for combinations of (a, b, r) that receive strong evidence from edge pixels.

```
[1]
✓ 0s
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
```

PART A — IMAGE UPLOAD & PREPROCESSING

```
[3]
✓ 10s
print("Upload an image for Hough Transform processing:")

uploaded = files.upload()

filename = list(uploaded.keys())[0]

img = cv2.imread(filename)

if img is None:
    print("Error: Image could not be loaded.")
else:
    print("Image uploaded successfully:", filename)
    print("Image size:", img.shape)
```

Upload an image for Hough Transform processing:
Choose Files Building.jpg
Building.jpg(image/jpeg) - 94045 bytes, last modified: 9/12/2026 - 100% done
Saving Building.jpg to Building (1).jpg
Image uploaded successfully: Building (1).jpg
Image size: (626, 1198, 3)

```
[4]
✓ 2s
plt.figure(figsize=(8, 6))

plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.show()
```

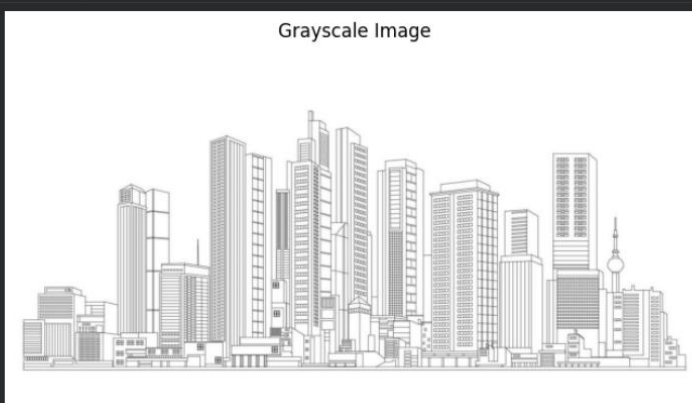


```
[5]
✓ 1s
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(8, 6))

plt.imshow(gray, cmap='gray')
plt.title("Grayscale Image")
plt.axis("off")

plt.show()
```



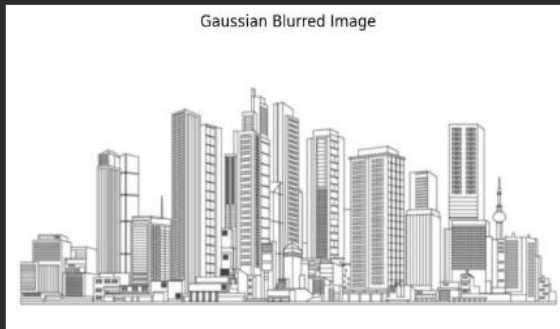
TASK 1 — PREPROCESSING

```
[6]
✓ Os
blur = cv2.GaussianBlur(gray, (5, 5), 0)

plt.figure(figsize=(8, 6))

plt.imshow(blur, cmap='gray')
plt.title("Gaussian Blurred Image")
plt.axis("off")

plt.show()
```

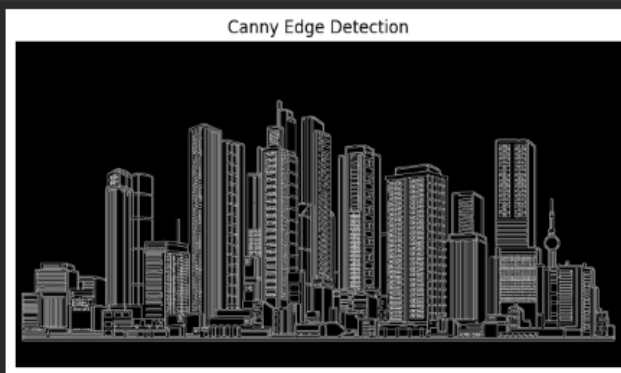


```
[7]
✓ Os
edges = cv2.Canny(blur, 50, 150)

plt.figure(figsize=(8, 6))

plt.imshow(edges, cmap='gray')
plt.title("Canny Edge Detection")
plt.axis("off")

plt.show()
```



```
[8]
✓ Os
plt.figure(figsize=(15, 4))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(edges, cmap="gray")
plt.title("Canny Edges")
plt.axis("off")

plt.tight_layout()
plt.show()
```



TASK 2 — HOUGH LINE DETECTION

```
[9] ✓ 1s
lines = cv2.HoughLines(
    edges,
    1,
    np.pi / 180,
    100
)

result_lines = img.copy()

if lines is not None:
    for line in lines:
        rho, theta = line[0]

        a = np.cos(theta)
        b = np.sin(theta)

        x0 = a * rho
        y0 = b * rho

        x1 = int(x0 + 1000 * (-b))
        y1 = int(y0 + 1000 * a)

        x2 = int(x0 - 1000 * (-b))
        y2 = int(y0 - 1000 * a)

        cv2.line(
            result_lines,
            (x1, y1),
            (x2, y2),
            (0, 0, 255),
            2
        )

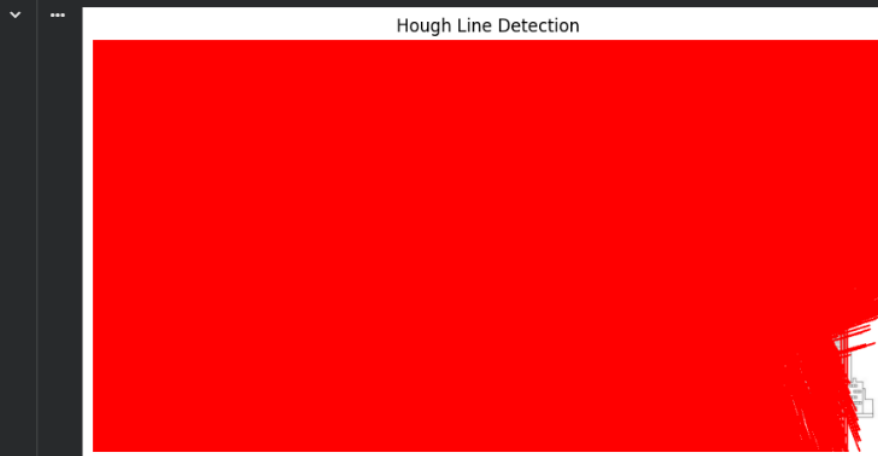
    print("Number of detected lines:", len(lines))
else:
    print("No lines detected.")
```

... Number of detected lines: 14351

```
[10] ✓ 0s
plt.figure(figsize=(10, 7))

plt.imshow(cv2.cvtColor(result_lines, cv2.COLOR_BGR2RGB))
plt.title("Hough Line Detection")
plt.axis("off")

plt.show()
```



TASK 2 — PARAMETER EXPERIMENT

```
[11] ✓ 2s
thresholds = [50, 75, 100, 150, 200]

plt.figure(figsize=(18, 10))

for i, threshold in enumerate(thresholds):
    lines = cv2.HoughLines(
        edges,
        1,
        np.pi / 180,
        threshold
    )

    result = img.copy()

    if lines is not None:
```

```
for line in lines:
    rho, theta = line[0]
    a = np.cos(theta)
    b = np.sin(theta)
    x0 = a * rho
    y0 = b * rho
    x1 = int(x0 + 1000 * (-b))
    y1 = int(y0 + 1000 * a)
    x2 = int(x0 - 1000 * (-b))
    y2 = int(y0 - 1000 * a)
    cv2.line(
        result,
        (x1, y1),
        (x2, y2),
        (0, 0, 255),
        2
    )

count = len(lines)

else:
    count = 0

plt.subplot(2, 3, i + 1)

plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Hough Threshold = {threshold}\nLines = {count}")
plt.axis("off")

plt.tight_layout()
plt.show()
```



PART B — HOUGH CIRCLE DETECTION

```
[12] ✓ Sm print("Upload an image containing circular objects:")
uploaded_circle = files.upload()
circle_filename = list(uploaded_circle.keys())[0]
circle_img = cv2.imread(circle_filename)
if circle_img is None:
    print("Error: Circle image could not be loaded.")
else:
    print("Circle image uploaded successfully:", circle_filename)
    print("Image size:", circle_img.shape)
```

Upload an image containing circular objects:
cars4.jpg (image/png) - 104223 bytes, last modified: 9/12/2026 - 100% done
Saving cars4.jpg to cars4.jpg
Circle image uploaded successfully: cars4.jpg
Image size: (736, 736, 3)

```
[13] ✓ Os plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(circle_img, cv2.COLOR_BGR2RGB))
plt.title("Original Circle Image")
plt.axis("off")
plt.show()
```



```
[14]
✓ On
circle_gray = cv2.cvtColor(
    circle_img,
    cv2.COLOR_BGR2GRAY
)

circle_blur = cv2.GaussianBlur(
    circle_gray,
    (9, 9),
    2
)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(circle_gray, cmap="gray")
plt.title("Circle Image - Grayscale")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(circle_blur, cmap="gray")
plt.title("Circle Image - Gaussian Blur")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Circle Image - Grayscale Circle Image - Gaussian Blur



```
[15]
✓ On
circles = cv2.HoughCircles(
    circle_blur,
    cv2.HOUGH_GRADIENT,
    dp=1,
    minDist=30,
    param1=100,
    param2=30,
    minRadius=10,
    maxRadius=100
)

circle_result = circle_img.copy()

if circles is not None:
    circles = np.uint16(
        np.around(circles)
    )

    print("Number of detected circles:", len(circles[0]))

    for circle in circles[0]:
        x, y, r = circle

        # Draw circle boundary
        cv2.circle(
            circle_result,
            (x, y),
            r,
            (0, 255, 0),
            2
        )

        # Draw center
        cv2.circle(
            circle_result,
            (x, y),
            2,
            (0, 0, 255),
            3
        )

    else:
        print("No circles detected.")
```

Number of detected circles: 80

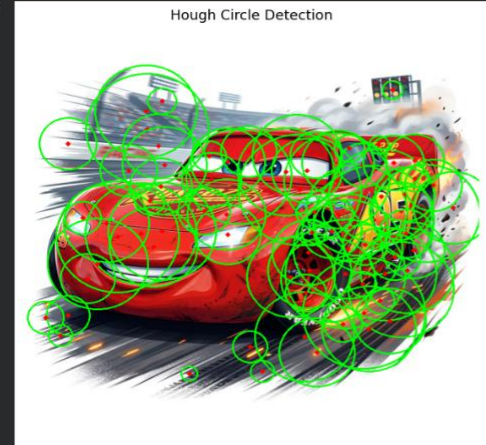
```
[16]
✓ On
plt.figure(figsize=(10, 7))

plt.imshow(
    cv2.cvtColor(circle_result, cv2.COLOR_BGR2RGB)
)

plt.title("Hough Circle Detection")
plt.axis("off")

plt.show()
```

Hough Circle Detection



```
[17] ✓ 7s
param2_values = [15, 20, 30, 40, 50]

plt.figure(figsize=(18, 10))

for i, p2 in enumerate(param2_values):

    circles_test = cv2.HoughCircles(
        circle_blur,
        cv2.HOUGH_GRADIENT,
        dp=1,
        minDist=30,
        param1=100,
        param2=p2,
        minRadius=10,
        maxRadius=100
    )

    result = circle_img.copy()

    if circles_test is not None:

        circles_test = np.uint16(
            np.around(circles_test)
        )

        for circle in circles_test[0]:
            x, y, r = circle

            cv2.circle(
                result,
                (x, y),
                r,
                (0, 255, 0),
                2
            )

            cv2.circle(
                result,
                (x, y),
                2,
                (0, 0, 255),
                3
            )

        count = len(circles_test[0])

    else:
        count = 0

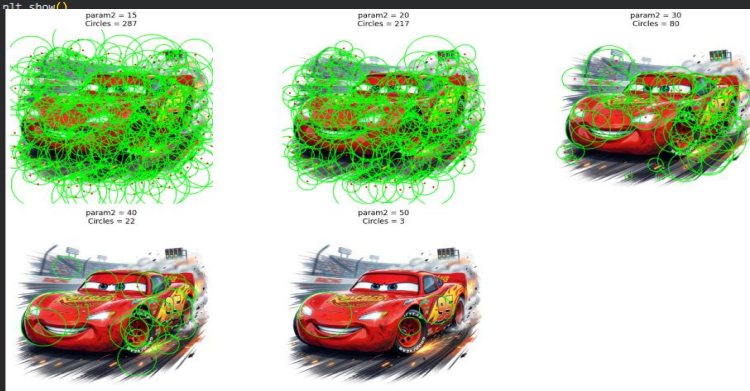
    plt.subplot(2, 3, i + 1)

    plt.imshow(
        cv2.cvtColor(result, cv2.COLOR_BGR2RGB)
    )

    plt.title(
        f"param2 = {p2}\nCircles = {count}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```



```
min_dist_values = [10, 20, 30, 50, 80]

plt.figure(figsize=(18, 10))

for i, distance in enumerate(min_dist_values):

    circles_test = cv2.HoughCircles(
        circle_blur,
        cv2.HOUGH_GRADIENT,
        dp=1,
        minDist=distance,
        param1=100,
        param2=30,
        minRadius=10,
        maxRadius=100
    )

    result = circle_img.copy()

    if circles_test is not None:
```



```

circles_test = np.uint16(
    np.around(circles_test)
)

for circle in circles_test[0]:

    x, y, r = circle

    cv2.circle(
        result,
        (x, y),
        r,
        (0, 255, 0),
        2
    )

    cv2.circle(
        result,
        (x, y),
        2,
        (0, 0, 255),
        3
    )

count = len(circles_test[0])

else:
    count = 0

plt.subplot(2, 3, i + 1)

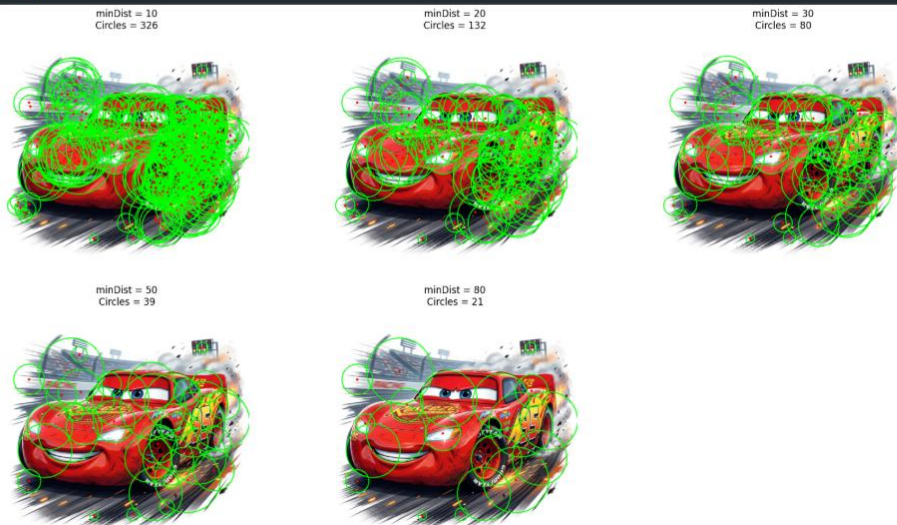
plt.imshow(
    cv2.cvtColor(result, cv2.COLOR_BGR2RGB)
)

plt.title(
    "minDist = (distance) \n Circles = (count)"
)

plt.axis("off")

plt.tight_layout()
plt.show()

```



```

19) ✓ Os
circles_radius = cv2.HoughCircles(
    circle_blur,
    cv2.HOUGH_GRADIENT,
    dp=1,
    minDist=30,
    param1=100,
    param2=30,
    minRadius=10,
    maxRadius=100
)

radius_result = circle_img.copy()

if circles_radius is not None:

    circles_radius = np.uint16(
        np.around(circles_radius)
    )

    for circle in circles_radius[0]:

        x, y, r = circle

        cv2.circle(
            radius_result,
            (x, y),
            r,
            (0, 255, 0),
            2
        )

        cv2.circle(
            radius_result,
            (x, y),
            2,
            (0, 0, 255),
            3
        )

    print(
        "Detected circles:",
        len(circles_radius[0])
    )

else:

    print("No circles detected.")

Detected circles: 80
TASK 5 — REAL-WORLD APPLICATION

[21] ✓ Os
# Detect circles again using selected parameters

circles_final = cv2.HoughCircles(
    circle_blur,
    cv2.HOUGH_GRADIENT,
    dp=1,
    minDist=30,
    param1=100,
    param2=30,
    minRadius=10,
    maxRadius=100
)

final_result = circle_img.copy()

if circles_final is not None:

    circles_final = np.uint16(
        np.around(circles_final)
    )

    circle_count = len(circles_final[0])

    for circle in circles_final[0]:

        x, y, r = circle

        # Circle boundary
        cv2.circle(
            final_result,
            (x, y),
            r,
            (0, 255, 0),
            2
        )

        # Circle center
        cv2.circle(
            final_result,
            (x, y),
            2,
            (0, 0, 255),
            3
        )

    print("Total circular objects detected:", circle_count)

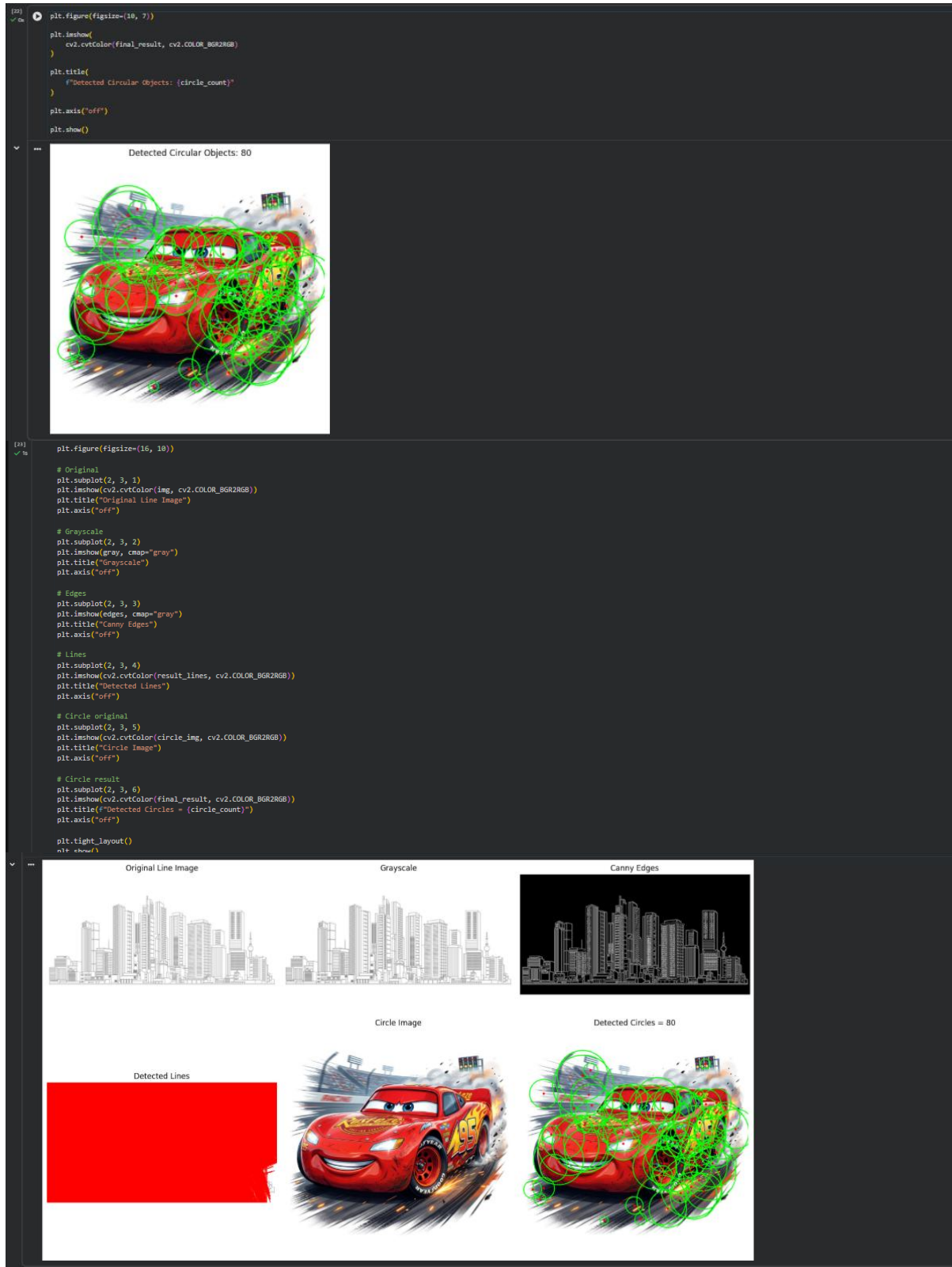
else:

    circle_count = 0

    print("No circular objects detected.")

... Total circular objects detected: 80

```



Question 1: Why is Gaussian filtering applied before edge detection?

Answer : Gaussian filtering is applied to reduce noise and small unwanted details in the image. Noise can create false edges during edge detection. Gaussian filtering makes the image smoother and helps produce cleaner edges for Hough Transform detection.

Question 2 : What happens if the Canny threshold values are too low?

Answer : If the Canny threshold values are too low, many weak and noisy edges are detected. This can produce unwanted edges and may result in false Hough line or circle detections.

Written Answers for the Assignment

Q1. Why is Gaussian filtering applied before edge detection?

Gaussian filtering reduces noise and small unwanted details from the image. Noise can create false edges during edge detection. Therefore, smoothing the image helps produce cleaner edges and improves Hough Transform detection.

Q2. What happens if Canny threshold values are too low?

Too many weak and noisy edges may be detected. These unwanted edges can result in false line or circle detections.

Q3. Why might a larger Gaussian kernel be useful for circle detection?

A larger Gaussian kernel provides more smoothing. It reduces noise and small unwanted edges, which can help the Hough Circle Transform detect circular boundaries more reliably.

Q4. What happens when the Hough threshold decreases?

More lines are detected because fewer votes are required for a line to be considered significant.

Q5. What happens when the Hough threshold increases?

Fewer lines are detected because only lines receiving a larger number of votes are considered significant.

Q6. Why do false lines appear?

False lines can occur due to noise, unwanted edges, textures, object boundaries, and an excessively low Hough threshold.